

XGBoost, From Scratch

A worked, hand-calculated example — similarity scores, gain, pruning, leaf weights, and how the tree is actually built.

$$\text{Similarity} = G^2 / (H + \lambda)$$

$$\text{Gain} = S_L + S_R - S_P$$

$$\text{Weight} = G / (H + \lambda)$$

How this walkthrough is structured

01

The Dataset

A tiny house-price table we will hand-calculate through the whole algorithm.

02

Initial Prediction

Start every model with one number: the mean.

03

Similarity Score

The formula XGBoost uses to score how 'pure' a group of residuals is.

04

Best Split & Gain

How the tree decides which question to ask first.

05

Pruning (γ)

Why some splits get cut even if they look useful.

06

Leaf Weights

Turning a leaf's residuals into an output value.

07

Updating Predictions

Learning rate η and building tree #2, #3, ...

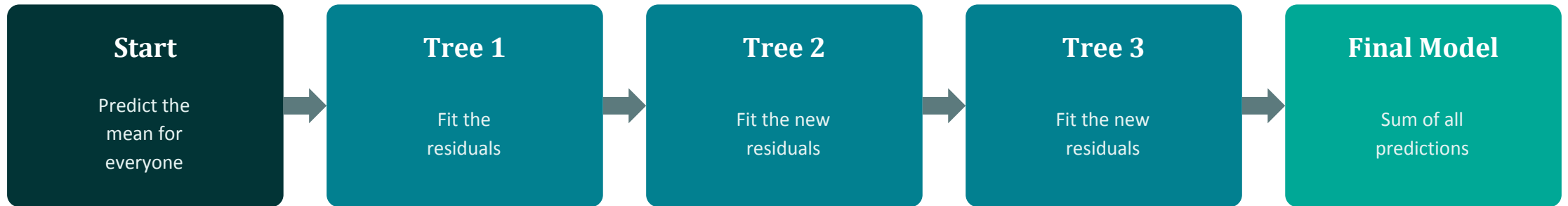
08

Classification Variant

Same machinery, new gradient & Hessian.

XGBoost = many weak trees, added together

XGBoost (Extreme Gradient Boosting) doesn't build one big tree. It builds trees **sequentially** — each new tree is trained to correct the **errors (residuals)** left behind by all the trees before it.



η (eta / learning rate) controls how much of each tree's correction gets added — small steps avoid overfitting.

Boosting

Sequential, error-correcting trees (unlike Random Forest's parallel trees)

Regularization

λ and γ keep trees simple and prevent overfitting

Additive Model

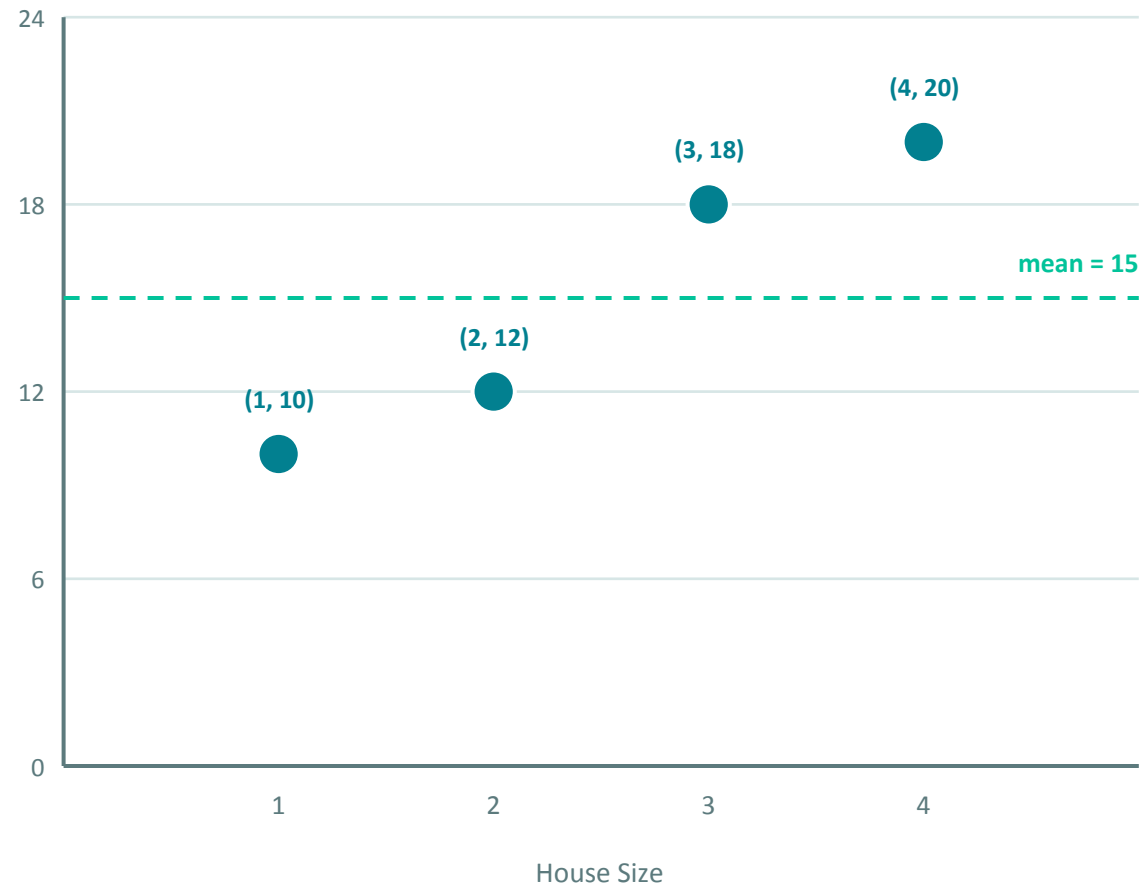
Final prediction = sum of every tree's contribution

The dataset: predicting house price from size

House Size	House Price (\$ '000s)
1	10
2	12
3	18
4	20

Just 4 rows, 1 feature — small enough to compute every formula by hand and see exactly what XGBoost is doing internally.

House Price vs. Size



STEP 1

Start with one number: the mean

Every XGBoost model begins with the same base prediction for every row — usually the **mean of the target**. This is the model before any trees exist.

$$\mu = (10 + 12 + 18 + 20) / 4 = 15$$

Residuals are simply the leftover error: $\text{Residual} = \text{Actual} - \text{Predicted}$. XGBoost's next tree will try to predict these residuals.

1

Size	Price	Prediction (μ)	Residual
1	10	15	-5
2	12	15	-3
3	18	15	+3
4	20	15	+5

Notice the residuals sum to $-5-3+3+5 = 0$ — the mean always balances out perfectly.

The Similarity Score: how XGBoost scores a group

$$\text{Similarity} = \frac{(\sum \text{Residuals})^2}{n + \lambda}$$

$\sum$ Residuals

Sum of residuals of every row inside this node

n

Number of rows (data points) in the node

λ (lambda)

Regularization term — higher λ shrinks similarity, discourages splits

Worked example — the parent node (all 4 rows)

Residuals in root node: -5, -3, +3, +5 (using $\lambda = 0$)

$$\text{Similarity} = \frac{(-5 - 3 + 3 + 5)^2}{4 + 0} = \frac{(0)^2}{4} = 0$$

A similarity of 0 makes sense: the positive and negative residuals cancel out, so this node looks like a poor, unspecialized grouping — it's a strong candidate to be split.

Finding the best split with Gain

Try candidate thresholds on House Size. For each one, split into a left / right child, score both, and compute the Gain of splitting versus not splitting.

$$\text{Gain} = \text{Similarity}_{\text{left}} + \text{Similarity}_{\text{right}} - \text{Similarity}_{\text{parent}}$$

Example split — House Size < 1.5

	Rows	Σ Residuals	n	Similarity
Left	{1}	-5	1	25
Right	{2, 3, 4}	+5	3	8.33

$$\text{Gain} = 25 + 8.33 - 0 = 33.33$$

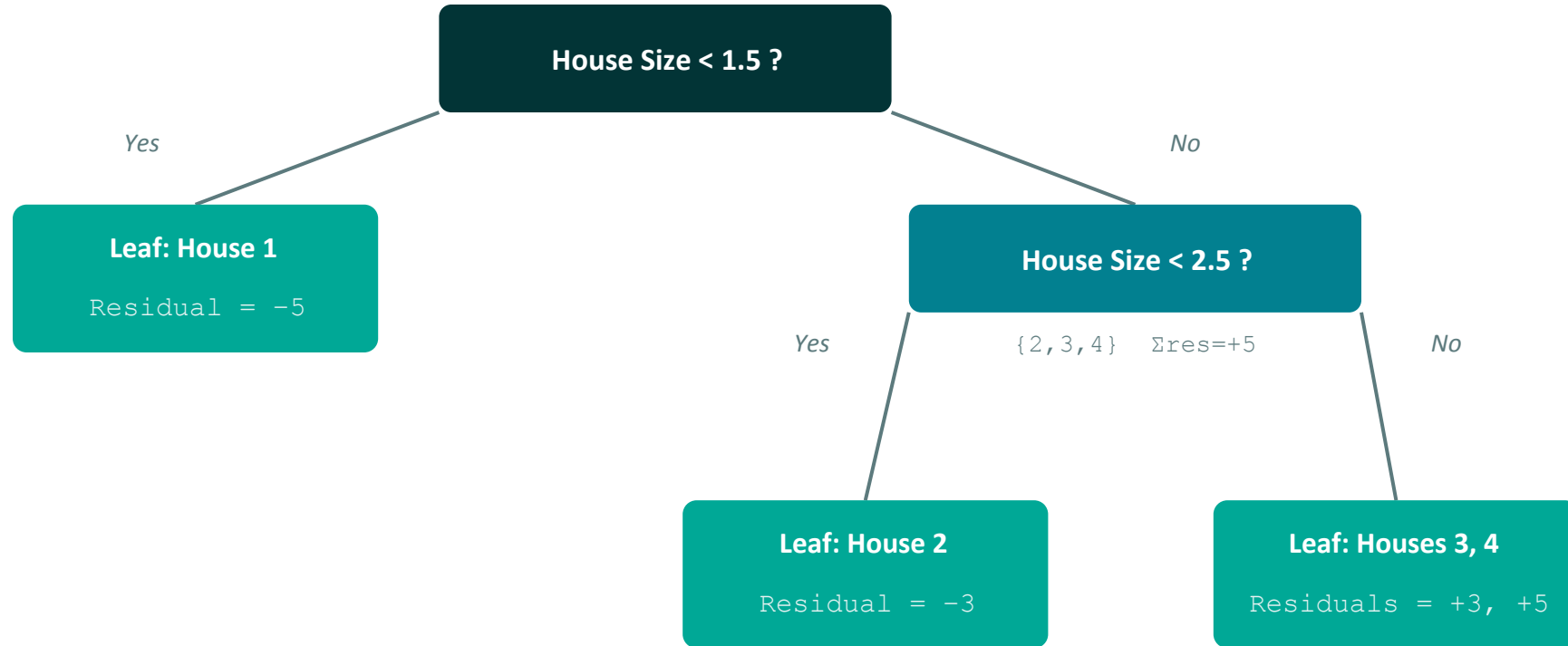
Every candidate is scored the same way — the highest Gain wins.

Split	Gain
Size < 1.5	33.33 ✓
Size < 2.5	—
Size < 3.5	—

(Other thresholds are evaluated exactly the same way — we picked Size < 1.5 for this walkthrough.)

Growing the tree, one split at a time

XGBoost repeats the same process inside each child node — recursively — until a stopping rule (max depth, or pruning) kicks in.



Splitting {2,3,4} at Size < 2.5: $\text{Sim}(\text{left } \{2\})=9$, $\text{Sim}(\text{right } \{3,4\})=32$, $\text{Sim}(\text{parent } \{2,3,4\})=8.33 \rightarrow \text{Gain} = 9+32-8.33 = \mathbf{32.67}$

Pruning: should this split actually happen?

A high Gain can still be rejected. XGBoost compares Gain against a minimum-gain threshold γ (gamma) before committing to a split — this keeps the tree from overfitting to noise.

$$\text{Pruning Value} = \text{Gain} - \gamma$$

Pruning value	Decision
Negative	Don't split — prune this node
Positive (or 0)	Split the node further

Worked example, using our Gain = 32.67 split

γ	$32.67 - \gamma$	Decision
20	+12.67	Split ✓
40	-7.33	Prune ✗

Why this matters for teaching

- A larger γ makes the tree more conservative — fewer splits, simpler trees.
- This is XGBoost's built-in defense against overfitting, done *during* tree construction (not just after, like standard pruning).
- Even if a split's Gain looks attractive on its own, XGBoost still weighs it against the cost γ of adding complexity.

Turning leaves into output values

Once splitting stops, every leaf gets a single output weight — the value this tree will contribute for any row that lands there.

$$\text{Leaf Weight} = G / (H + \lambda)$$

G = sum of residuals (gradients) in the leaf

H = number of rows in the leaf (Hessian = 1 per row for squared-error loss)

λ = regularization term — same as before

Leaf	Rows	G (Σ residuals)	H (n)	Weight = G/(H+λ)
House 1	1	-5	1	-5.00
House 2	1	-3	1	-3.00
Houses 3 & 4	2	+8	2	+4.00

These three numbers (-5, -3, +4) are Tree 1, complete — the finished output of our first boosted tree.

Tree 1 output: House 1 → -5 House 2 → -3 Houses 3,4 → +4

Updating predictions with the learning rate

$$\text{New Prediction} = \mu + \eta \times (\text{Tree Weight})$$

η (eta) is the learning rate — how much of each tree's correction we actually apply. We'll use $\eta = 0.3$ (XGBoost's typical default).

Small η = many small steps toward the right answer

Large η = big jumps — faster, but risks overshooting and overfitting

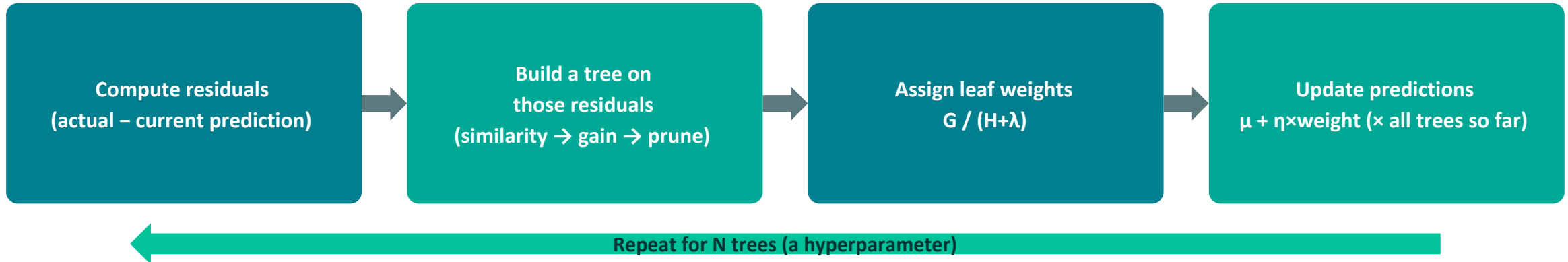
This is exactly why boosting needs *many* trees — each one only nudges the prediction a little.

Size	Price	New Pred.	New Resid.
1	10	13.5	-3.5
2	12	14.1	-2.1
3	18	16.2	+1.8
4	20	16.2	+3.8

$$\text{New Pred.} = \mu + 0.3 \times (\text{tree weight})$$

The residuals shrank (e.g. $-5 \rightarrow -3.5$) but didn't hit zero — **Tree 2** will now be built on **these new residuals**, using the exact same steps 2–6.

The boosting loop — repeat until done



Final model

$$\text{Final Prediction} = \mu + \eta \times \text{Tree}_1 + \eta \times \text{Tree}_2 + \eta \times \text{Tree}_3 + \dots + \eta \times \text{Tree}_N$$

Each tree is small and imperfect on its own — together, added up with a small learning rate, they form a strong, accurate model.

Same machinery, for classification

For a classification target (Pass / Fail), the formulas keep their shape — only the gradient (G) and Hessian (H) definitions change.

Study Hrs	Result	y	Initial p	Residual (y-p)
1	F	0	0.5	-0.5
2	F	0	0.5	-0.5
3	P	1	0.5	+0.5
4	P	1	0.5	+0.5

Every prediction starts at $p = 0.5$ (log-odds of 0), same idea as starting at the mean.

$$\text{Similarity} = \frac{(\sum \text{Residuals})^2}{\sum [p(1-p)] + \lambda}$$

Gradient: $g = p - y$ (loss derivative)

Hessian: $h = p(1-p)$ (from log-loss)

Leaf weight: $w = G / (H + \lambda) \rightarrow$ converted to a probability via sigmoid

Root similarity check: $\sum \text{residuals} =$
 $-0.5 - 0.5 + 0.5 + 0.5 = 0$
 $\sum p(1-p) = 4 \times 0.25 = 1 \rightarrow \text{Similarity} = 0^2 / (1 + 0)$
 $= 0$

Formula cheat-sheet

Similarity Score

$$S = G^2 / (H + \lambda)$$

Gain of a split

$$\text{Gain} = S_{\text{left}} + S_{\text{right}} - S_{\text{parent}}$$

Pruning rule

$$\text{if } (\text{Gain} - \gamma) < 0 \rightarrow \text{prune}, \text{ else} \rightarrow \text{split}$$

Leaf output weight

$$w = G / (H + \lambda)$$

Updated prediction

$$\hat{y}_{\text{new}} = \hat{y}_{\text{prev}} + \eta \times w$$

Regression G, H

$$G = \Sigma (\hat{y} - y) \quad H = n \quad (1 \text{ per row})$$

Classification G, H

$$G = \Sigma (p - y) \quad H = \Sigma p(1 - p)$$

λ = L2 regularization on leaf weight • γ = minimum gain to allow a split • η = learning rate

Questions?

From a mean, to residuals, to similarity, to gain, to a pruned tree of leaf weights — that's XGBoost, one formula at a time.